LET'S PLAY TOGETHER
CANNON GAME
START

Computer Programming, Algorithms and Data Structures, Mod. 1

September 24, 2025

Final Exam Project

Daria Miele (mat. 535702)

Academic Year 2023/2024

GENERAL OVERVIEW

A 2D physics-based artillery game developed using Python and Kivy.

GAME

AIM AND FIRE PROJECTILES FROM THE CANNON TO HIT THE TARGET ON THE OPPOSITE SIDE

STRUCTURE

HIT THE TARGET WITH LIMITED ATTEMPTS

GOAL

10 ROUNDS OF INCREASING DIFFICULTY

5 ATTEMPTS PER ROUND TO HIT THE TARGET

PROJECTILE TYPES

THERE ARE 3 DIFFERENT TYPES OF PROJECTILES

BULLET

- Massive projectile
- Subject to gravity
- Parabolic trajectory
- Limited impact radius

BOMBSHELL

- Massive projectile
- Subject to gravity
- Parabolic trajectory
- Larger explosion radius

LASER

- Massless
- Not affected by gravity
- Linear trajectory
- Limited duration

BE AWARE OF OBSTACLES

ROCK

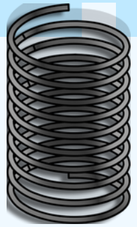
Destructible ground element

BULLETPROOF MIRROR

Reflects lasers, can move in later rounds

PERPETIO

Indestructible, block projectiles

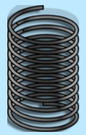


ELASTONIO

Makes bullet/bomb bounce, animates in later rounds

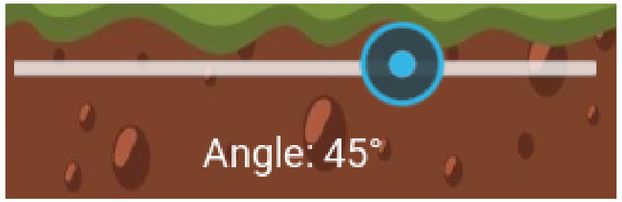
PROGRESSIVE DIFFICULTY

Obstacles appear progressively, increasing in number and complexity round after round, making the game more challenging

Rounds	Obstacles Introduced
1-2	Rock
3	+ Perpetio
4-5	+ Mirror
6	+ MOVING Mirror
7-8	+ Elastonio 
9-10	+ MOVING Elastonio

PLAYER CONTROLS

The player controls the cannon to aim precisely and hit the target.



Slider to adjust launch angle

Slider to adjust muzzle velocity

Space-bar to shoot

FUNCTIONALITIES

Key features to enhance the user's experience

HELP

Provide basic instructions to guide the player

HALL OF FAME

Persistent leaderboard to register best scores

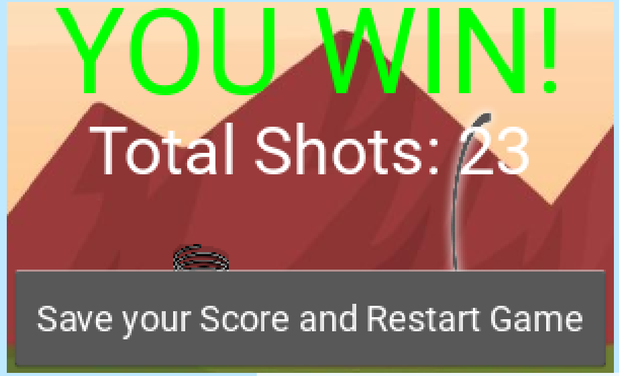
SAVE

Store round, attempts and selected projectiles for each match

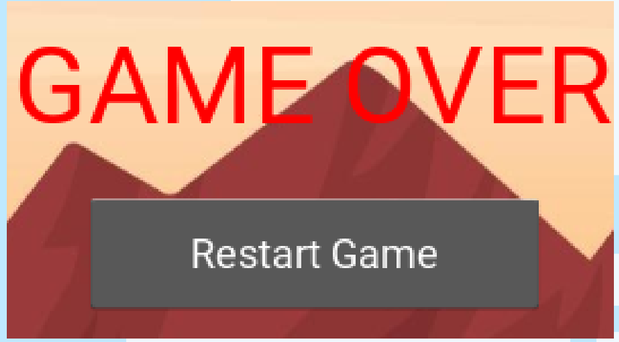
PROGRESS AND ENDGAME

A status label displays current round and attempts

Round: 7 - Attempt: 2/5



At the end, labels and buttons show results and allow restart or saving in the Hall of Fame



GAME FLOW

Welcome

Projectile Selection

Gameplay

Win

Save score in Hall of Fame

Game Over (Lose)

REMEMBER

In each moment, you can return to the Projectile Selection, Help and Hall of Fame screens!

IMPLEMENTATION OVERVIEW

Modular design: each entity in its own class

Parameters defined as constants

Screen size, projectile physics, gravity

```
SCREEN_WIDTH = 1000
SCREEN_HEIGHT = 700
FPS = 20
BULLET_MASS = SCREEN_WIDTH/2
BOMB_MASS  = SCREEN_WIDTH/3
BULLET_MAX_VEL = BULLET_MASS
BOMB_MAX_VEL = BOMB_MASS
LASER_VEL = SCREEN_WIDTH/1.5
BULLET_RADIUS = SCREEN_WIDTH/70
BOMB_RADIUS = SCREEN_WIDTH/50
LASER_DIST = SCREEN_WIDTH/100
BOMB_DRILL = SCREEN_WIDTH/50
LASER_IMPULSE = 3
```

Game loop with update(dt)

handles physics, collision and game flow

MAIN CLASSES

CannonApp	Entry point, manages ScreenManager
CannonGame	Core logic of the game
Projectile	Unified class, manages physics for bullet, bomb, laser
Obstacle	Each obstacle has its own class RockObstacle, PerpetioObsatcle, MirrorObstacle, ElastonioObstacle

CannonApp(App)

Application entry point

- Creates the ScreenManager, which handles navigation across different screens
- Instantiates all screens: Welcome, ProjectileSelect, Game, Help, HallOfFame
- Returns the root widget to start the app

Key method

build() → create screens, add them to manager and return manager

```python
class CannonApp(App):
    def build(self):
        self.manager = ScreenManager()
        welcome_screen = WelcomeScreen(name="welcome")
        self.manager.add_widget(welcome_screen)
        projectile_select_screen = ProjectileSelectScreen(name="projectile_select")
        self.manager.add_widget(projectile_select_screen)
        game_screen = CannonGame(name="game")
        self.manager.add_widget(game_screen)
        help_screen = HelpScreen(name="help")
        self.manager.add_widget(help_screen)
        hof_screen = HallOfFameScreen(name="hall_of_fame")
        self.manager.add_widget(hof_screen)
        return self.manager
```

CannonGame (Screen)

Core gameplay controller and main loop

- Manages round state, attempts, score, save/load
- Spawns target and obstacles per round

```
def new_round(self)
```

```
def spawn_obstacles(self):
```

- Handles input (sliders + spacebar) and fires shots

```
def fire_projectile(self):
```

- Runs the loop: move projectiles/obstacles, collisions, win/lose

```
def update(self, dt)
```

Projectile (Image)

Physics + visual behaviour for Bullet, Bomb, Laser

Encapsulates all projectile mechanics without duplicating code

- Stores type, vx/vy, angle, time, gravity flag
- Updates motion each frame

```
def move(self, dt)
```

- Triggers visuals (bomb explosion, laser rotation)
- Supports special states (reflected once on mirrors)

Obstacles (Image)

Each obstacle is its own class (Image), same interface

- RockObstacle: destructible ground tile `def on_hit(self)`
- PerpetioObstacle: indestructible ground tile `def on_hit(self, projectile)`
- MirrorObstacle: reflects lasers; optional horizontal motion `def update(self, dt)`
- ElastonioObstacle: elastic bounce for bullets/bombs, absorbs laser; optional vertical motion `def on_hit(self, projectile)` `def update(self, dt)`

FUTURE IMPROVEMENTS

Visual design

Add custom graphics, animations and sound effects to make the gameplay more immersive

Enhanced obstacle behavior

More complex obstacles behaviors could be added

Gameplay variety

Introduce new projectile mechanics or obstacle types

Advanced scoring system

The score could also consider accuracy, time taken, or bonus points for special shots

GAME IS OVER!

THANK YOU

COMPUTER PROGRAMMING, ALGORITHMS AND DATA STRUCTURES, MOD.1

DARIA MIELE (535702)

ACADEMIC YEAR 2023/24

Developing the game was both a technical challenge and a learning experience. The project required to combine principles of object-oriented programming, physics simulation, and user interface design within the Kivy framework. At the end, a functional and extensible solution was created.